Binary Exploitation:

An Introduction to Exploit Development

TechX

Exploit development is the process by which we take advantage of a vulnerability, bug, or design flaw in a system. The primary objective is to cause unintended behavior, most commonly *arbitrary code execution.*

Binary Exploitation refers specifically to exploit development targeting *compiled binaries*.

This workshop focuses on exploiting *memory corruption* vulnerabilities in compiled C/C++ programs.

Binary Exploitation Steps

01

Vulnerability Identification

02

Execution Flow Hijacking

03

Mitigation Bypasses

04

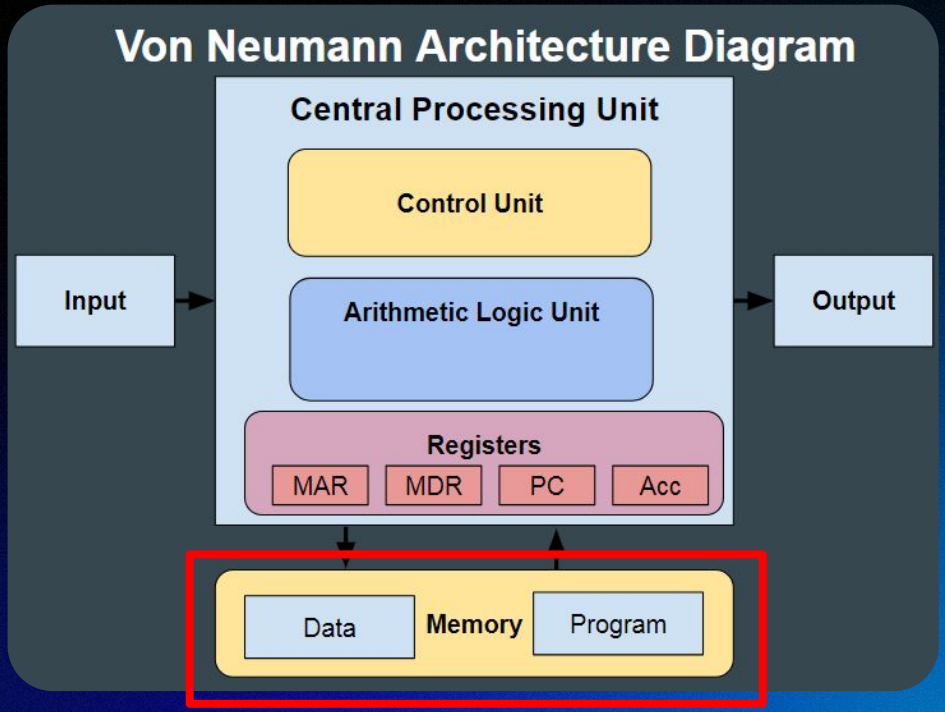
Payload Engineering

05

Weaponization and Delivery

The Original Sin: Von Neumann Architecture

- Nearly every exploit resulting in ACE has been the product of this architectural choice
  - **Data** (User input) can manipulate sensitive **program memory**
  - All of the progress we have made (OS level mitigations, memory safe languages, etc.) have aimed to correct this fundamental flaw

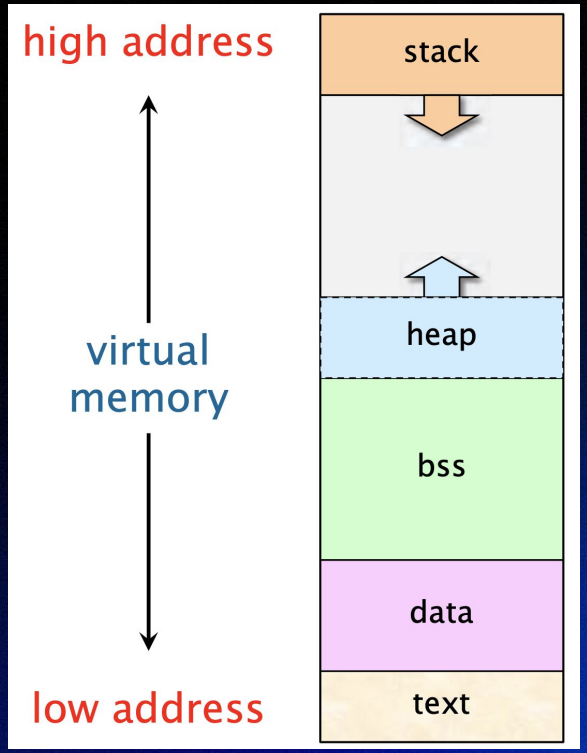


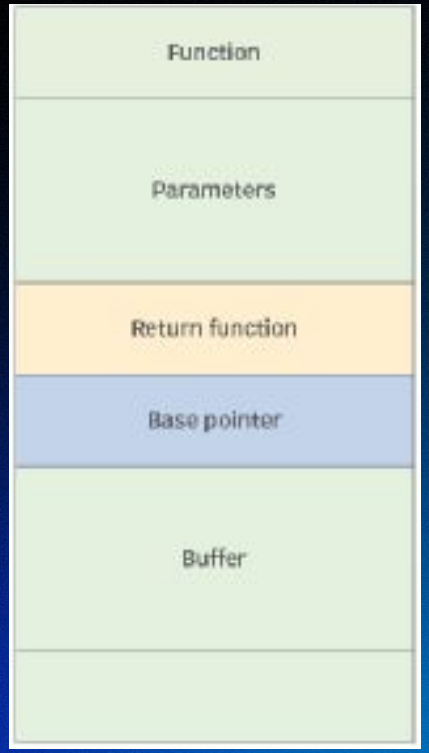
What *bugs* might exist which would allow a users input to modify program memory?

Recall: Stack Behavior

- **Abstraction:** Segment memory into stack, heap, data, etc.
  - We will focus specifically on memory corruption bugs that target *stack memory*

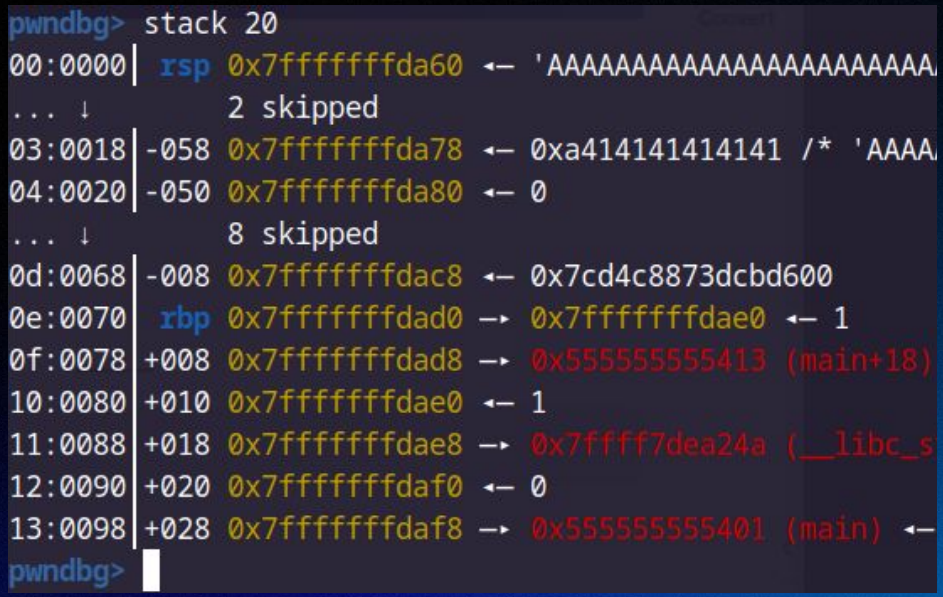
- New stack frame created each function call
  - Stack frames grow down in memory
  - Memory grows up within frames





Recall: Stack Behavior

- Your machine must keep track of the *instruction* it is currently executing
  - It does this by storing an *instruction pointer*
  - %rip for intel x86-64
- When a function finishes it **returns** to the instruction following the function call
  - This *return address* is stored on the stack, along with misc. data
  - RA is loaded into the instruction pointer and execution jumps there



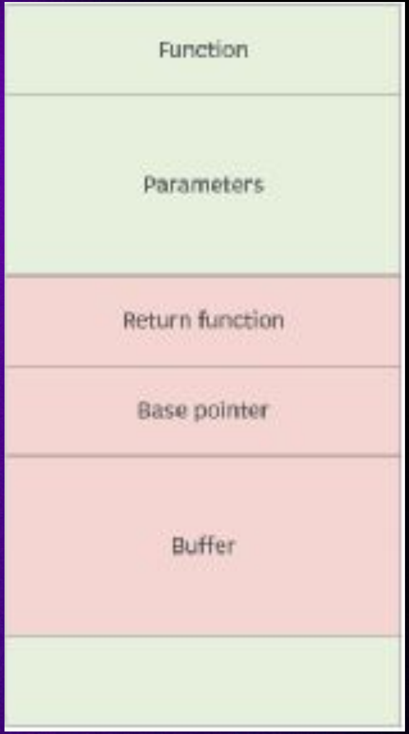
Smashing the Stack for Fun and Profit

- **Buffer Overflow** occurs when the programmer fails to limit the size of an input that gets stored in a fixed memory buffer
  - This allows an attacker to fill the entire stack with arbitrary data
  - Redirect execution flow by modifying the return address

```
Starting program: /home/owen/Desktop/CTFs/picoCTF/binEx/buffer_overflow_2/vuln
warning: Unable to find libthread_db matching inferior's thread library, thread debugging will not be available.
Please enter your string:
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```c
void vuln(){
  char buf[BUFSIZE];
  gets(buf);
  puts(buf);
}
```

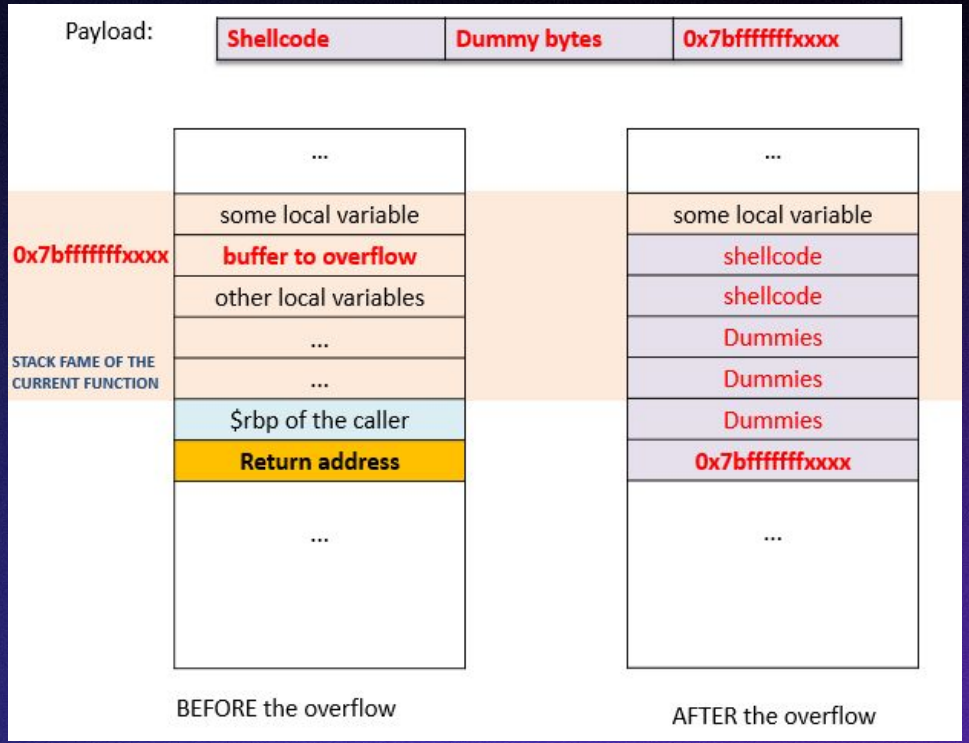
```
pwndbg> c
Continuing.
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

Program received signal SIGSEGV, Segmentation fault.
0x41414141 in ?? ()
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA

*EAX  0x1ed
*EBX  0x41414141 ('AAAA')
*ECX  0xf7f9a9b8 ◂— 0
 EDX  1
 EDI  0xf7ffcb80 (_rtld_global_ro) ◂— 0
 ESI  0x80493f0 (__libc_csu_init) ◂— endbr32
*EBP  0x41414141 ('AAAA')
*ESP  0xffffcce0 ◂— 0x41414141 ('AAAA')
*EIP  0x41414141 ('AAAA')

Invalid address 0x41414141
```



Lab 1 – Ret2Win

Gaining RCE

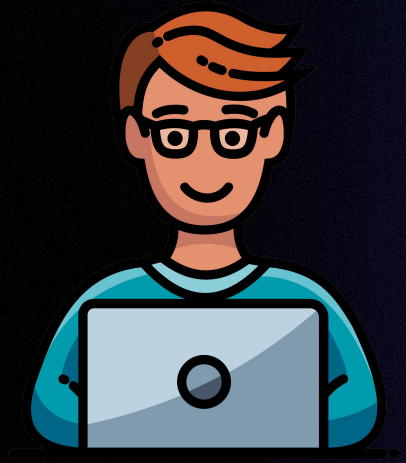
- *Recall:* Our goal is to execute arbitrary code
  - With a buffer overflow vulnerability, we can make the program jump *anywhere*
- **Shellcode:** We can take a small snippet of compiled, executable code and supply that code as an input to the vulnerable buffer
  - We can then modify the return address s.t. the instruction pointer jumps to code we supplied

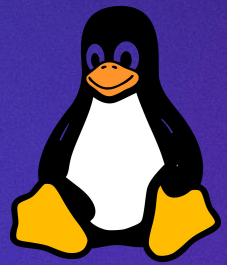


Lab 2 – Code Execution

This seems a little *too easy*…

Whose responsibility is it to prevent such vulnerabilities?





Mitigations

Responsibility is Shared

Developers have a responsibility to write secure software, but mistakes are inevitable.

As a result, we build a number of system level mitigations into our respective operating systems and compilers.

Non Executable (NX) Stack

An attacker shouldn't be able to simply inject malicious code, point the instruction pointer at that code, and execute it. We mark pages in memory that get written to as non executable. Instructions can *only* be executed from memory regions that are *executable*.

ASLR

Each of our exploits have relied on us being able to address things within the program at runtime. We found those addresses by reverse engineering the binary. Address Space Layout Randomization randomizes these addresses, so we can't trivially reference specific memory regions in our exploit.

Stack Canaries

Notice, in order to modify the return address we had to overwrite everything on stack. We can make such an attack more difficult by adding a random value to our stack, before the return address. If that random value gets overwritten, the program will know it is being exploited, and it can halt execution.

Well that was fun i guess no more exploitation everything is secure YAY!!

NO!!!

Recall: Binary Exploitation Steps

01 **Vulnerability Identification**

02 **Execution Flow Hijacking**

03 **Mitigation Bypasses**

04 **Payload Engineering**

05 **Weaponization and Delivery**

In practice, we never use just a *single vulnerability…*

Chaining Primitives

Instead, we treat standalone vulnerabilities as *exploit primitives*. By chaining several of these primitives we can construct an exploit that bypasses common mitigations.

Arbitrary Write

Arbitrary Write, formally known as a **write-what-where** condition allows an attacker to *write* an arbitrary value to an arbitrary location.

Examples:

- OOB Write
- Use-After-Free (UAF)
- Externally controlled format string

Relative Write

Relative Write conditions allow an attacker to control sensitive program information via memory corruption from a specific location. Victim information must be a fixed offset from the write location.

Examples:

- Stack buffer overflow

Arbitrary Read

Arbitrary Read, formally known as a **read-what-where** condition allows an attacker to *read* an arbitrary value from memory.

Examples:

- Externally controlled format string
- UAF

Relative Read

Relative Read conditions allow us to read an a value in memory, at a fixed offset from some vulnerable location in memory.

Examples:

- Missing null terminator
- Improper bounds check

Format Strings

- In C, **printf()** is a function that can be used to print some string
  - The first parameter takes in raw text and format specifiers
  - The subsequent parameters replace the format specifiers in the raw text
- A vulnerability occurs when an attacker controlled string is passed in as the first parameter
  - The attacker adds additional format specifiers, resulting in an arbitrary read / write

printf("My age is %d",23); output: My age is 23
where %d is format specifier for integer

```
fgets(buffer, 64, stdin);
printf(buffer);
```

```
┌─[✗]─[owen@parrot]─[~/Desktop/fstrings_practice]
└──╼ $./format1
%p.%p.%p.%p.%p.%p.
0x55e7105482a1.0xfbad2288.0x7fe39f2a429d.0x55e7105482b3.0x21001.0x7ffd199ba258.
```

Bypassing Mitigations – ASLR

- *Recall:* ASLR makes exploitation harder by randomizing the address space our program executes in
  - Memory addresses change each time the program executes
  - Offsets between different addresses *remain constant*
- If we leak a *single address* from our program, we can calculate every other address in a given region of memory

- STEPS:
  - Use a read primitive to leak some memory address
    - F-strings
  - Using reverse engineering techniques, find the offset between that address we can leak, and the target address
  - Leak memory and calculate the target address (leak + offset) at runtime

Bypassing Mitigations – Stack Canary

- *Recall:* Stack canaries are random values that sit on the stack, allowing the program to detect buffer overflows before we overwrite the return address
- Using a read primitive we can leak the stack canary, and then overwrite it with itself

- STEPS:
  - Use read primitive to leak stack canary
    - F-strings
  - Construct a payload that fills the buffer from the beginning of the buffer to the start of the canary, add the canary to that payload, and then pad until the return address.
    - AAAA…AcanaryAAAA…A return_addr

Bypassing Mitigations – NX

- Notice, we can't trivially bypass this mitigation with an additional primitive
- If we can't execute shellcode on the stack, can we still execute arbitrary code?
- *IDEA:* We use "gadgets" within the program itself to execute arbitrary code, using a technique called Return Oriented Programming (ROP)

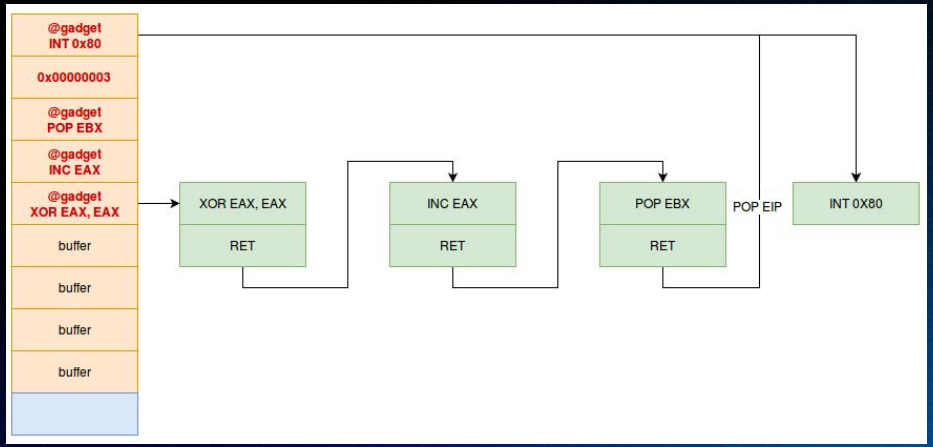
- ROP:
  - In a given codebase there will exist snippets of assembly that look something like this:

    mov rax, rsi;
    ret;

  - These snippets can be chained to achieve ACE
  - We place the respective addresses for our chain of instructions on the stack, and then when we redirect the IP to the start of our chain

ROP Illustrated

- *Recall:* The **ret** instruction just pops an address off the top of the stack, and sets the instruction pointer to that address
- Hence, when we place a sequence of addresses on the stack that correspond to a single instruction followed by a return, each instruction will execute and then return directly to the next



Lab 3 – ACE with Mitigations Enabled

Thank you!